Technical Design Document

Table of Contents Links

[Technical Design Document](#)
- [To Do Before Next Tuesday:](#)
- [Stack](#)
- [TABLES FOR DATABASE](#)
- [UI Pages](#)
  - [Admin](#)
  - [Driver](#)
  - [Instructor](#)
  - [Student](#)
  - [Chatbot](#)
  - [Login](#)
  - [Client](#)

(click refresh to update)

To Do Before Next Tuesday:

- ☐ Pragun: and Lakshya: Backend + Api
- ☐ Krishal and Kenton: Frontend

Stack

COMPONENT	PROPOSED TECHNOLOGY	INTENDED USE
Frontend	React and TypeScript	Application screens and interactions
Build tool	Vite	All in one tool that provides direct build tool
Interface styling	Tailwind CSS with shadcn/UI components	Creates for simple and easy styling, similar to native web but enhanced
Frontend hosting	Cloudflare Pages	Hosting the frontend
Backend API	Cloudflare Worker	Request validation, permissions, database operations and AI requests

Database	PostgreSQL hosted on Neon	Relational data storage and SQL reporting
Database administration	pgAdmin 4	Schema inspection and query development
AI service	Gemini API, provisionally	Natural-language interpretation and response generation
Version control	GitHub	Source management and team collaboration
Development environment	VS Code	Application and database development

TABLES FOR DATABASE

TABLES	PURPOSE AND MAIN RELATIONSHIP	Columns
Student	Stores student information; linked to package purchases and lessons	studentID (PK), firstName, lastName, email, phone, defaultPickupAddress (optional), registrationDate, status
Instructor	Stores instructor information; linked to lessons and availability	instructorID (PK), firstName, lastName, email, phone, status
Service	Defines lesson or service types and durations	serviceID (PK), serviceName, description, licenceClass, durationMinutes, basePrice, isActive
Package	Defines package offerings and included allowances	packageID (PK), serviceID (FK), packageName, numberOfLessons, packagePrice, isActive

PackagePurchase	Links a student to a purchased package and preserves purchase terms	purchaseID (PK), studentID (FK), packageID (FK), purchaseDate, lessonsPurchased, agreedPrice, expiryDate (optional), status
Lesson	Links a student, instructor and service; optionally linked to a package purchase	lessonID (PK), studentID (FK), instructorID (FK), serviceID (FK), purchaseID (FK, optional), startDateTime, endDateTime, pickupAddress, status, countsTowardPackage, bookingNotes (optional)
InstructorAvailability	Records instructor availability periods	availabilityID (PK), instructorID (FK), startDateTime, endDateTime
LessonFeedback	Stores authorized feedback linked to a lesson	feedbackID (PK), lessonID (FK), authorUserID (FK), strengths, areasToImprove, nextLessonFocus, createdAt
ApplicationUser	Supports authenticated access and user roles	userID (PK), authProviderUserID, role, instructorID (FK, optional, unique), isActive, createdAt

UI Pages

Admin

The dashboard should show a weekly summary of new students and lessons booked, along with pending approval requests. The navigation menu should include Dashboard, Students, Staff, Bookings, Packages, Approval Requests, Reports and Payments. The admin can manage student and staff records, lesson bookings and packages. They can approve schedule-change

requests from drivers, special timing or group-session requests from students, and late cancellation or rescheduling requests. The Payments page should allow the admin to record payments received outside the app and track outstanding balances.

Driver

The dashboard should show today's lessons, including student names, lesson times and pickup locations. A weekly calendar should also be available to view assigned lessons. After each lesson, the driver can update attendance and add feedback about skills practised and areas that need improvement. Drivers can request changes to their schedules, but these changes require admin approval. The navigation menu should include Dashboard, Weekly Schedule, Student Feedback and Schedule Requests.

Instructor

The instructor's page should support individual sessions and group classes based on requests. Instructors should be able to view their assigned sessions and the students registered for each session. After a session, they can record attendance, topics covered, practice-test scores and individual feedback on areas to improve. The navigation menu should include Dashboard, Schedule, Attendance, Practice-Test Scores and Student Feedback.

Student

The dashboard should show only upcoming sessions and remaining package lessons. Students can book available slots immediately, while special timings or group sessions require admin approval. They can cancel or reschedule before the school's cutoff; changes after the cutoff require admin approval. A separate My Progress page should display instructor feedback and practice-test scores, while attendance records should appear on a separate Attendance page. The navigation menu should include Dashboard, Book a Session, My Bookings, My Progress, Attendance and Chatbot.

Chatbot

The chatbot should answer questions about the school's services, prices and policies. Logged-in students should also be able to ask about their own upcoming bookings and remaining package lessons. Personal information should be limited to the student's own records. For the initial version, the chatbot will provide information; booking and schedule changes will be handled through the relevant application pages.

Login

Client

2 different viewpoints
2 different dashboards for both consumers and
Login Page
Desktop: Schedule, reminder chatbot, Map location at the bottom (current location tracking)

AI Integration

Use one shared chatbot backend with two interfaces: a student assistant and an instructor assistant. Both use the same PostgreSQL data, but the backend restricts which records and functions each user can access.

Your proposal originally treats student login as optional. This plan adds student access, so confirm that scope with the client before implementation.

AI integration plan: LAC Driving School

1. Purpose and scope

The chatbot will help users retrieve lesson and package information through ordinary language. Gemini interprets the question, while the Cloudflare Worker validates permissions and executes approved SQL queries against Neon PostgreSQL.

The initial version will **retrieve information and help users navigate to booking screens**. Booking changes will use the application's normal forms and validation.

	Student assistant	**Instructor assistant**
Main purpose	Understand personal bookings, package balances, and available lessons	Review teaching schedules and information needed for assigned lessons
Interface location	Student dashboard and booking page	Instructor dashboard and lesson detail page
Suggested questions	"When is my next lesson?" "How many lessons do I have left?" "Find a lesson next Friday afternoon."	"Show my schedule tomorrow." "Summarize this student's lesson history." "Which of my lessons were cancelled this week?"
Data access	The logged-in student's records, available slots, and approved service information	The instructor's schedule and client-approved information about assigned students
Results	Lesson cards, package balance summaries, available-slot cards	Schedule lists, workload summaries, authorized student-history summaries

Actions	Open booking details or a prefilled booking form	Open lesson details or a feedback form

School-wide reports, revenue, and other instructors' student records remain outside instructor access unless the client explicitly authorizes them.

2. Interface design

Build a reusable React `ChatPanel` component with role-specific titles, suggested questions, and result cards.

Student interface

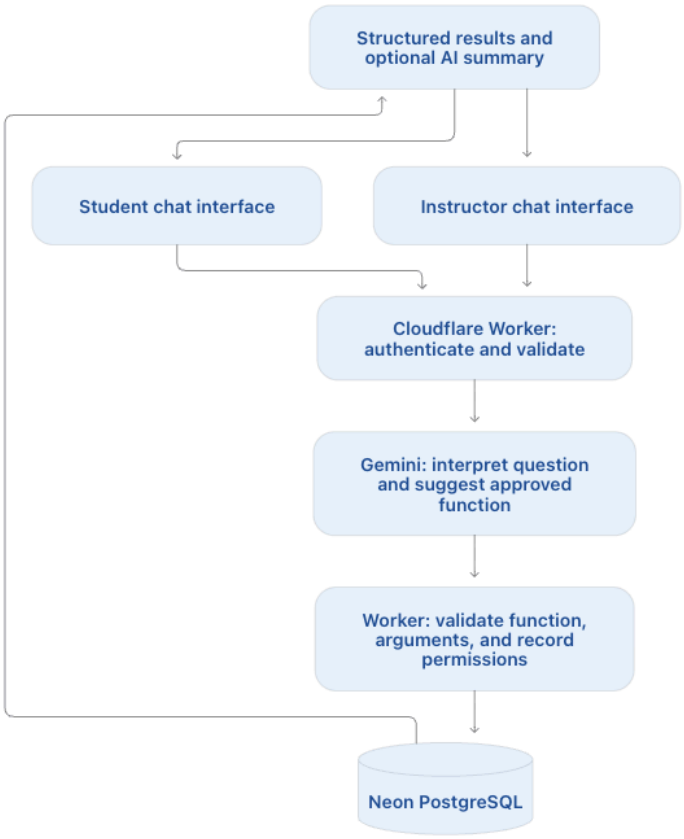
- Title: **Lesson Assistant**
- Suggested buttons: **My next lesson**, **My package balance**, **Find availability**
- Answers link to the relevant booking or package record.
- Available lessons include a **Review booking** button that opens the normal booking form.

Instructor interface

- Title: **Instructor Assistant**
- Suggested buttons: **Today's schedule**, **This week's summary**, **Selected student's history**
- Student-specific questions use an explicitly selected student or ask the instructor to select one.
- Private instructor notes and student-visible feedback are distinguished.

Both interfaces show loading, retry, empty-result, and unavailable-service states. Display dates in the school's agreed timezone and label live results with their retrieval time.

3. Backend architecture



Use **Gemini function calling**: the model proposes a named operation and its parameters; your backend decides whether to execute it. Gemini supports connecting models to application functions and database-backed operations this way. [Gemini function-calling documentation]

Suggested approved functions:

Function	Authorized use	Main tables
`get_my_upcoming_lessons`	Student's own bookings	`Student`, `Lesson`, `Instructor`, `Service`
`get_my_package_balance`	Student's own purchases and usage	`PackagePurchase`, `Package`, `Lesson`

`find_available_slots`	Student search using valid service/date filters	`InstructorAvailability`, `Lesson`, `Service`
`get_my_teaching_schedule`	Instructor's own schedule	`Instructor`, `Lesson`, `Student`, `Service`
`get_assigned_student_history`	Instructor with an authorized teaching relationship	`Student`, `Lesson`, `LessonFeedback`
`get_my_lesson_summary`	Instructor's own workload and cancellation totals	`Instructor`, `Lesson`

The server derives the user's identity and role from authentication. It must not trust a student ID, instructor ID, or role supplied by the chatbot or browser. Every function independently checks authorization.

The model receives no database credentials and cannot submit arbitrary SQL.

4. Database preparation

Use the existing proposed tables, with these details clarified:

- Link `ApplicationUser` to the appropriate `Student` or `Instructor` record.
- Preserve the allowance and purchase terms in `PackagePurchase`.
- Define feedback visibility, for example `student_visible` versus `instructor_only`.
- Store lesson times consistently and convert them to the school's timezone for display.
- Define which statuses consume package entitlement, including cancellations and no-shows.

Create reporting views for **package balances**, **lesson schedules**, and **instructor workload**. PostgreSQL calculates balances and totals; the chatbot explains the results.

For example, a six-lesson purchase with two completed and one scheduled should show:

> Four lessons remain, including one already booked.

That calculation must follow the client's agreed consumption rules.

No vector database or full chat-history database is needed initially. Optional request logs can record operation name, duration, success, and token usage without retaining complete conversations.

5. Example interaction

Student: "Can I book a lesson next Friday afternoon?"

1. The assistant resolves the date and asks for clarification if needed.
2. Gemini suggests `find_available_slots` with validated search filters.
3. The Worker queries instructor availability and existing lessons.
4. The interface displays actual available slots.
5. The student selects **Review booking**.
6. The booking endpoint rechecks availability and package eligibility inside a database transaction before saving.

The chatbot finding a slot does not reserve it.

For instructor feedback, the assistant can later help draft wording, but the instructor must review and save it. Automatically marking lessons completed or altering package balances is outside the initial AI scope.

6. Cost, privacy, and failure handling

Use a Gemini model available on the free API tier, or an existing funded API key. Keep the key in Worker secrets and make the model configurable.

Suggested initial limits are **10 chat requests per user per day**, short responses, and at most two database-function calls per message. These are application limits your team controls, separate from provider quotas.

Use client-approved representative or de-identified data for demonstrations. Gemini's free tier currently lists content use for product improvement; a paid key changes the applicable terms and costs. Gemini pricing and data-use information

If AI is unavailable, normal booking, package, and reporting screens must still work. The assistant should report the failure rather than invent results.

7. Implementation and verification

Stage	Deliverable
1. Confirm requirements	Client approves student access, instructor permissions, feedback visibility, and package rules
2. Build database operations	Tested parameterized SQL functions and reporting views
3. Add AI interpretation	Gemini selects only approved functions with validated arguments
4. Build both interfaces	Shared chat component with student and instructor configurations
5. Test and review	Client demonstrations, documented feedback, revisions, and retesting

Required test cases should include:

- A student cannot retrieve another student's records.
- An instructor cannot access an unrelated student or another instructor's schedule.
- Package balances match SQL results.
- Ambiguous dates trigger clarification.
- Empty results and API failures produce understandable messages.
- Concurrent booking attempts cannot reserve the same instructor time.
- Instructions embedded in messages or feedback cannot bypass permissions.

Recommended course-project scope: deliver personal lesson/package search for students and schedule/history reporting for instructors first. Add booking preparation and feedback drafting only after those core features work and the client has reviewed them.

L.A.C. Driving School – Business and Website Analysis

Course: CMPT 339 – Introduction to Database Management Systems
Client: L.A.C. Driving School
Website: https://lac-driving-school.square.site/
Date Reviewed: October 26, 2026

1. Business Overview

L.A.C. Driving School is a driving instruction business operating in British Columbia, Canada. The school provides professional driving lessons for beginner, intermediate, and experienced drivers who want to improve their driving skills and prepare for their road tests.

The school offers training for Class 5, Class 7, and Class 4 Restricted driver's licences. Lessons are conducted by a certified driving instructor.

The main services include:

- Beginner driving lessons
- Intermediate driving lessons
- Refresher driving courses
- Class 5 and Class 7 licence training
- Class 4 Restricted licence training
- Road test preparation
- Road test packages
- Vehicle rental for road tests
- Pre-trip inspection practice
- Multiple-lesson packages

The school's main objective is to help students improve their driving abilities, build confidence, and prepare for safe driving and road tests.

2. Existing Website and Booking System

The school currently uses a website created through Square Online.

The website provides information about the school, available lessons, prices, operating hours, pickup locations, and policies.

It also includes a **Book Appointment** option connected to Square's appointment-booking service.

The existing website has the following main sections:

1. Home – Business information, services, and pricing.
2. Book Appointment – Access to the appointment-booking system.
3. Hours – Business operating hours.
4. Pick-Up Locations – Designated student pickup points.
5. School Policies – Rules regarding lessons, payments, cancellations, and refunds.

The website provides a way for customers to book lessons, but the school's internal methods for tracking student packages, completed lessons, remaining sessions, and instructor schedules need to be confirmed with the client.

3. Driving Lessons and Pricing

A. Individual Driving Lessons

Service	Duration	Price (CAD)
Class 5 and 7 assessment session	1 hour	$70
Class 5 and 7 driving lesson	1.5 hours	$100
Class 5 and 7 driving lesson	2 hours	$130
Class 4 Restricted lesson (pre-trip included)	1.5 hours	$150

B. Road Test Packages

These packages include a one-hour warm-up before the road test.

Service	Total Duration	Price (CAD)
Class 5 and 7 road test package	2.5 hours	$200
Class 5 and 7 road test package	2 hours	$180
Class 4 road test package	3 hours	$250

C. Multiple-Lesson Packages

Each lesson in these packages is 1.5 hours long.

Package	Total Hours	Price (CAD)	Advertised Savings
4 Lessons	6 hours	$390	$10
6 Lessons	9 hours	$580	$20
8 Lessons	12 hours	$770	$30
10 Lessons	15 hours	$960	$40
12 Lessons	18 hours	$1,150	$50

D. Additional Services

Service	Duration	Price (CAD)
Class 5 and 7 vehicle rental	1 hour	$100
Pre-trip practice only	30 minutes	$50
Additional observer during a lesson	Per permitted observation	$20

These services and packages show why the proposed database may need to store different lesson types, prices, durations, package purchases, and remaining lesson balances.

4. Business Operating Hours

Day	Operating Hours
Monday	9:00 AM – 6:00 PM
Tuesday	9:00 AM – 6:00 PM
Wednesday	9:00 AM – 6:00 PM
Thursday	9:00 AM – 6:00 PM
Friday	9:00 AM – 6:00 PM

Saturday	10:00 AM – 5:00 PM
Sunday	10:00 AM – 5:00 PM

These hours could be considered when developing the instructor availability and lesson scheduling components of the proposed system.

However, individual instructor availability and scheduling outside regular business hours must be confirmed with the client.

5. Student Pick-Up Locations

The school provides designated pickup locations across Langley, Surrey, Burnaby, Vancouver, and Coquitlam.

Langley

- Willowbrook Mall (ICBC) – 19950 Willowbrook Drive, Langley Township, BC

Surrey

- Guildford Mall – 10355 152 Street, Surrey, BC
- Surrey Central Station – 10321 City Parkway, Surrey, BC

Burnaby

- Home Depot near Gilmore Station – 3950 Henning Drive, Burnaby, BC
- Patterson SkyTrain Station – 4101 Beresford Street, Burnaby, BC

Vancouver

- Joyce SkyTrain Station – 5099 Joyce Street, Vancouver, BC

Coquitlam

- Lincoln SkyTrain Station – 1140 Pinetree Way, Coquitlam, BC

According to the website, student pickup is restricted to the designated locations.

For the database project, these locations could be stored in a separate PickupLocation table and linked to lesson bookings.

This would allow the school to identify where each student should meet the instructor.

6. School Policies and Business Rules

The school's published policies establish several important rules that may influence the database design.

Driver's Licence Requirements

- Students must carry a valid British Columbia driver's licence during in-car lessons.

Lesson Duration

- The policies state that regular in-car lessons have a minimum duration of 90 minutes.
- Each 90-minute lesson includes at least 80 minutes of driving time.
- If an instructor arrives late, the student must receive the full lesson time or have the missing time added to a future lesson.
- If a student arrives late, additional lesson time is not provided.

Lesson Participation

- The school primarily provides one-on-one driving instruction.
- One additional observer may attend for $20.
- A maximum of three people, including the instructor and student, may be inside the vehicle during a lesson.

Payments

- Lessons must be paid for in full before they begin.
- The school does not accept installment payments.
- Package upgrades are not accepted.

Lesson Packages

- Purchased lessons and packages cannot be transferred to another person.
- Package lessons are expected to be completed within 12 months of payment.
- If lessons remain incomplete after 12 months, the school does not provide a refund or additional lessons.

Cancellation and Rescheduling

- Students must provide at least 48 hours' notice when cancelling a lesson.
- The published policy specifies a $30 fee when the required notice is not provided for rescheduling.
- Students who do not attend a scheduled lesson without notice forfeit that lesson.

Refunds

- All lesson and package purchases are final sales.
- Purchased lessons and packages are non-refundable and non-transferable.

Road Test Results

- The school does not guarantee that students will pass their road tests or obtain a driver's licence.

These policies are important because the proposed database should accurately represent lesson attendance, cancellations, payments, package validity, and remaining sessions.

7. Proposed Database System for CMPT 339

Based on the existing services and booking process, our team proposes developing a PostgreSQL-based management system for L.A.C. Driving School.

The main purpose would be to organize student information, instructor schedules, lesson records, and package usage in one centralized database.

The following are proposed functions, not yet confirmed client requirements.

7.1 Student Management

The system could maintain student records containing:

- Student ID
- Full name
- Contact information
- Driver's licence class
- Registration date
- Purchased lesson packages
- Lesson history

Each student would have a unique identification number to connect their information with lessons and packages.

7.2 Instructor Management

The system could store:

- Instructor ID
- Instructor name
- Contact information
- Availability
- Assigned lessons

- Teaching schedule

This would help prevent instructors from being assigned overlapping appointments.

7.3 Lesson Scheduling and Management

The system could maintain records containing:

- Lesson ID
- Student ID
- Instructor ID
- Lesson type
- Scheduled date
- Start and end times
- Lesson duration
- Pickup location
- Lesson status

Possible lesson statuses include Scheduled, Completed, Cancelled, Rescheduled, and No-Show.

7.4 Package Management

Students may purchase packages containing several lessons.

The proposed system could record:

- Package ID
- Package name
- Package price
- Number of lessons included
- Purchase date
- Expiration date
- Completed lessons
- Remaining lessons

For example, if a student purchases a six-lesson package and completes two lessons, the system could show that four lessons remain.

The exact rules for counting and deducting lessons must be confirmed with the client.

7.5 Payment Records

If payment tracking is included in the approved project scope, the database could record:

- Payment ID

- Student ID
- Related package or lesson
- Payment amount
- Payment date
- Payment status

The system would need to respect the school's full-payment and no-installment policies.

7.6 Reports and Data Analysis

The system could generate reports showing:

- Upcoming scheduled lessons
- Completed lessons
- Cancelled lessons and no-shows
- Student lesson history
- Remaining package sessions
- Packages approaching expiration
- Instructor schedules
- Lesson demand by type
- Payment summaries, if authorized

These reports would help the school monitor its daily activities and make operational decisions.

8. Proposed Database Tables

The initial relational database design could include:

Table	Purpose
Student	Stores student profiles and contact details
Instructor	Stores driving instructor information
Lesson	Stores individual driving lesson records
LessonType	Stores service types, durations, and prices
Package	Stores available lesson packages
StudentPackage	Records packages purchased by students

PickupLocation	Stores approved student pickup locations
InstructorAvailability	Stores available instructor times
Payment (optional)	Stores authorized payment records
ApplicationUser	Stores authorized system users and roles

The tables would be connected using primary keys and foreign keys.

For example, the Lesson table would reference Student, Instructor, and PickupLocation to identify who is attending the lesson, who is teaching it, and where it begins.

The final table structure will depend on the client's requirements and the agreed project scope.

9. Proposed AI Integration

As part of the CMPT 339 project requirements, our team plans to include a meaningful AI feature connected to the database.

One possible feature is a natural-language reporting assistant.

Instead of manually writing SQL queries, an authorized user could ask questions such as:

- "How many lessons are scheduled this week?"
- "Which students have only two lessons remaining?"
- "Show all completed lessons this month."
- "Which instructor has the most upcoming bookings?"
- "Which lesson packages are expiring soon?"

The AI component could interpret these questions and provide answers using information retrieved from the PostgreSQL database.

The AI feature should operate within the user's access permissions and avoid exposing unnecessary personal information.

10. Important Questions for the Client

Before finalizing the database design, our team needs clarification on the following points:

1. How does the school currently maintain student records and lesson histories?
2. Does Square Appointments already manage all instructor schedules and bookings?
3. How are completed and remaining package lessons currently tracked?
4. Is a lesson deducted when it is booked or only after it is completed?

5. How are cancelled lessons, late cancellations, and no-shows recorded?
6. Can a student purchase more than one package at the same time?
7. Does the school want payment records included in our database?
8. How many instructors will use the proposed system?
9. Which employees should be authorized to access and modify records?
10. What reports would be most useful to the owner or administrator?
11. Should the new system work separately from Square or exchange information with it?
12. Are there any existing records that the school can share in anonymized form for development and testing?

Two published details also require clarification: the minimum 90-minute lesson rule versus the shorter services advertised on the homepage, and how the different road test packages are selected and charged.

11. Conclusion

L.A.C. Driving School provides several types of driving lessons, road test preparation services, and multi-lesson packages across multiple pickup locations.

Its existing Square Online website allows customers to access booking services and learn about prices, locations, hours, and policies.

For our CMPT 339 project, a centralized relational database could help organize student information, instructor availability, lesson scheduling, and package usage.

Using PostgreSQL, the proposed application would establish relationships between students, instructors, lessons, and purchased packages while supporting useful operational reports.

The planned AI integration could further assist authorized staff in retrieving and understanding information from the database through natural-language questions.

The final system requirements will be refined through discussions with the client to ensure the application addresses the school's actual operational needs.

References

L.A.C. Driving School. (2026). *Official website and driving lesson information.* https://lac-driving-school.square.site/

L.A.C. Driving School. (2026). *Hours of operation.* https://lac-driving-school.square.site/hours

L.A.C. Driving School. (2026). *Pick-up locations.* https://lac-driving-school.square.site/pick-up-locations

L.A.C. Driving School. (2026). *School policies.* https://lac-driving-school.square.site/policies

L.A.C. Driving School. (2026). *Online appointment booking.* https://lac-driving-school.square.site/s/appointments

Project: LAC Driving School (Proposal)

Team B – GPT

Trinity Western University

CMPT 339: Introduction to Database Management

Andrew Park

October 12, 2026

TEAM MEMBERS, SPECIALITIES AND ROLES

m member	ıdent No.	Email Address	of responsibility	Project Role
Bell		.bell@mytwu.ca	ation development egration	Developer
Gupta		.Gupta@mytwu.ca	nal database design L	se Developer
a Sai T.		ayayaa@gmail.com	e design, usability ordination	eader and UI/UX er
Patel		.patel@mytwu.ca	gration and tion testing	gration and Testing per

Kenton will lead application implementation and connect the interface to the backend API. Pragun will design and implement the PostgreSQL database, including relationships, integrity constraints, queries and reporting views. Lakshya will coordinate the team and design the application's screens and user workflows. Krishal will integrate the AI feature and coordinate functional and integration testing. All members will participate in requirements analysis, client review and documentation.

BACKGROUND

LAC Driving School offers different driving services, including individual lessons, lesson packages, driving assessments, pre-trip practice, and road-test services. Based on its public booking website, students can purchase Class 5/7 lesson packages with four, six, eight, ten, or twelve lessons. Students are asked to book their first lesson online and arrange their next lessons through email or by talking to their instructor. The school also uses Square for online bookings, where students can check available times and enter their contact information.

This process gives us an opportunity to study how the school manages student packages and driving lessons. For example, the school may need to track which package a student purchased, how many lessons they have completed, and how many lessons they have left. However, we do not yet know if the school has any problems with its current system or if Square already handles these tasks.

Our proposed project will focus on managing student packages and driving lessons, depending on what the school confirms. We plan to learn how LAC Driving School currently stores and manages this information, what tasks are handled outside Square, and whether a PostgreSQL database application could help staff organize records, track lessons, and prepare reports more easily.

Problem to Be Investigated (until confirmed):

The exact problem has not yet been confirmed by LAC Driving School. Our team plans to investigate how the school manages student lesson packages, bookings, and completed lessons. After speaking with the client, we will confirm which tasks need improvement and whether a PostgreSQL database application would be helpful.

Client representative and participation:

Our team plans to interview a representative from LAC Driving School and review sample records approved by the client. We will use the information collected to understand the school's needs and develop a prototype. We will then ask the client to test the main features, collect their feedback, and make changes where needed before the final submission. The client's representative and level of participation are still to be confirmed.

PURPOSE AND VALUE

The purpose of the proposed system is to provide a central application for managing student records, package purchases and lesson activity. It will connect these records in PostgreSQL and present them through an interface that does not require users to write SQL.

Subject to the client's confirmed needs, the application will help staff identify remaining lesson entitlements, review student histories and monitor upcoming or incomplete bookings. It will distinguish completed lessons from future reservations so that package balances remain understandable.

Useful reports and an embedded AI database assistant will support information retrieval. For example, authorized staff could identify students who have unused lessons but no upcoming appointment. The intended value is more consistent record keeping and easier access to operational information.

Success will be evaluated through accurate package calculations, enforcement of agreed business rules, useful reporting and client verification of the main workflows.

TARGET USER AND STAKEHOLDERS

The proposed primary users are the school's owner or administrator and driving instructors, subject to confirmation.

The owner or administrator will manage student records, services, package purchases and lesson assignments, and access operational reports. Instructors will view assigned lessons and record completion and feedback where authorized. Students will benefit from accurate lesson records and package balances; direct student access will be considered only if requested and feasible.

LAC Driving School is the client stakeholder and will help establish requirements and evaluate the prototype. The project team and course instructor are additional stakeholders responsible for implementation and academic assessment.

Role-based permissions will control access. Final permissions will be agreed with the client so that users can access only the records and operations appropriate to their responsibilities.

DETAILED DESCRIPTION OF THE PROPOSED DATABASE APPLICATION

APPLICATION SCOPE

We propose a responsive web application focused on student package purchases, lesson records and operational reporting. Its initial scope will support an administrative workflow and instructor access.

The school's existing Square appointment service will inform requirements gathering. The prototype will initially support manual entry or an approved data import. Automatic Square integration will be considered only after confirming technical access, client permission and development feasibility.

MAJOR FUNCTIONS

Student management

We will let staff add, search and update student profiles. Each profile will show the student's packages and lesson history in one place. We will only collect the contact details the school actually needs.

Service and package management

We will let the owner set up the school's services, lesson lengths and packages, such as a six-lesson package. When a student buys a package, we will save the number of lessons and the price they paid at that time, so later price changes will not affect past purchases.

Package purchase tracking

We will record every package a student buys separately, so students can buy more than one package. We will link each lesson to the package that paid for it, and keep it separate from lessons paid for individually.

Lesson management

We will let staff book lessons, assign instructors, and set lesson times and statuses. We plan to use four statuses: scheduled, completed, cancelled and no-show, and we will adjust these

based on what the school needs. We will also stop the app from double-booking a student or an instructor at the same time. If the school keeps using Square for bookings, we will clearly document how the two systems relate so the records do not conflict.

Lesson balance calculation

We will have the app calculate how many lessons each student has left, and show upcoming bookings separately. For example, if a student buys six lessons, completes two and has one booked, the app will show four lessons left, with one already booked. This example assumes a lesson is used up once it is completed. We will confirm with the school how cancelled and missed lessons should be counted.

Instructor feedback

If the school wants it, we will let instructors write notes about each lesson and the student's progress. We will attach these notes to the specific lesson and student so they are easy to find later.

Data operations

We will let staff add, update and delete records where it makes sense. We will protect important history, such as past lessons and purchases, from being deleted by accident by cancelling or archiving those records instead. Only the development team or an administrator will be able to change the database structure.

A student buys a package, an instructor teaches the lessons, and the app keeps count of how many lessons the student has left.

PRELIMINARY RELATIONAL DATABASE DESIGN

TABLES	PURPOSE AND MAIN RELATIONSHIP
Student	Stores student information; linked to package purchases and lessons
Instructor	Stores instructor information; linked to lessons and availability
Service	Defines lesson or service types and durations

Package	Defines package offerings and included allowances
PackagePurchase	Links a student to a purchased package and preserves purchase terms
Lesson	Links a student, instructor and service; optionally linked to a package purchase
InstructorAvailability	Records instructor availability periods
LessonFeedback	Stores authorized feedback linked to a lesson
ApplicationUser	Supports authenticated access and user roles

Each table will have an appropriate primary key. Foreign keys will maintain relationships, including links between lessons and students, instructors, services and package purchases.

Integrity rules will require valid lesson times, positive package allowances and nonnegative monetary amounts. The system will also validate that a package-funded lesson belongs to the student who purchased that package and prevent consumption beyond the permitted allowance.

Transactions will protect related updates. For example, completing a lesson and recording its entitlement consumption must produce a consistent result even when multiple users work simultaneously.

The final schema, record volume and test dataset will be established after reviewing client requirements. We will use real client-related data where permitted, or anonymized or representative records approved by the client.

EMBEDED AI FEATURE

The core AI feature will be a conversational database assistant for authorized staff. Users will ask operational questions in ordinary language, such as:

- Which lessons were cancelled this week?
- Show the lesson summary for a selected student.
- Which students have remaining package lessons but no upcoming booking?

The AI will interpret the request and select an approved reporting operation. The backend will validate the user's permissions and query parameters, execute parameterized SQL against PostgreSQL and return structured results. The assistant will then present a readable response.

The AI will not execute unrestricted SQL or independently change bookings. Package balances and report totals will be calculated by PostgreSQL. Ambiguous requests will require clarification, and unavailable information will be identified rather than invented.

The Gemini API is the provisional provider. API credentials will remain in backend secrets. The prototype will use anonymized or representative AI inputs, and the final provider and data-handling arrangements will be confirmed before using client information. Personalized recommendations are an optional extension, dependent on a specific client need.

REPORTING VIEWS AND EXPECTED OUTPUTS

VIEWS OR REPORT	EXPECTED OUTPUT
Student package balance	Purchased allowance, consumed entitlement, remaining lessons and future bookings
Lesson schedule	Lessons by date, student, instructor, service and status
Instructor workload	Scheduled and completed lessons by instructor and period
Cancellation summary	Cancelled and missed lessons within a selected period
Student lesson history	Past lessons, statuses and authorized feedback
Follow-up report	Students with unused entitlement and no upcoming lesson

A dashboard will summarize selected operational measures, such as upcoming lessons, completed lessons and students requiring follow-up. Users will be able to search and filter records through the interface.

REQUIREMENTS AND TECHNOLOGIES

COMPONENT	PROPOSED TECHNOLOGY	INTENDED USE
ontend	eact and TypeScript	pplication screens and interactions
uild tool	te	l in one tool that provides direct build ol
terface styling	ilwind CSS with shadcn/UI mponents	reates for simple and easy styling, milar to native web but enhanced
ontend hosting	oudflare Pages	osting the frontend
ackend API	oudflare Worker	equest validation, permissions, atabase operations and AI requests
atabase	ostgreSQL hosted on Neon	elational data storage and SQL porting
atabase administration	gAdmin 4	chema inspection and query evelopment
service	emini API, provisionally	atural-language interpretation and sponse generation
ersion control	itHub	ource management and team ollaboration
evelopment nvironment	S Code	pplication and database evelopment

The frontend will communicate with the Worker API. The Worker will enforce authorization and access PostgreSQL; database credentials will not be exposed to the browser.

The prototype is expected to operate within available free-tier quotas at low traffic. This is a budget assumption rather than a guarantee. Hosting limits, AI model availability and potential charges will be checked during implementation.

The application will require authenticated access, validated inputs and secure credential storage. PWA installation is intended to improve access on supported devices; live database operations will still require connectivity. Offline booking is outside the initial scope.

TIMELINE AND DEVELOPMENT MILESTONES

PERIOD	ACTIVITIES AND DELIVERABLES
Week 1	Confirm client participation, interview the client and document the current workflow
Week 2	Finalize scope, business rules, approved dataset and interface sketches
Week 3	Complete ER diagram and relational schema; review the proposed workflow with the client
Week 4	Implement PostgreSQL tables, constraints, reporting views and test data
Week 5	Build student, package purchase and lesson interfaces; connect the backend API
Week 6	Implement package calculations, reports, permissions and the AI assistant
Week 7	Conduct functional and integration testing; demonstrate the prototype to the client
Week 8	Obtain client testing and verification, document feedback, revise and retest
Final Milestone	Complete the application, final report and presentation by the confirmed course deadlines

CONTRIBUTION IN PROPOSAL

Kenton Bell

- Led the main project idea discussions.
- Contributed to planning how the proposed system would work.

Pragun Gupta

- Contributed to pitching and discussing the project idea.
- Took the initiative to speak with the TA about finding a client.
- Prepared and organized the proposal documentation.

Lakshya Sai Tamminana

- Took the initiative to speak with the TA about finding a client.
- Rechecked and reviewed the proposal.
- Created and added visual diagrams to the document.

Krishal Patel

- Contributed to pitching and discussing the project idea.
- Attended the project discussion.
- Agreed with the proposed project idea and plan.

REFRENCES

CMPT 339. *Team Project Proposal Instructions*. Course handout supplied by the instructor.

LAC Driving School. *Appointments and Services*. https://lac-driving-school.square.site/s/appointments

PostgreSQL Global Development Group. *PostgreSQL Documentation*.

React. *React Documentation*.

Cloudflare. *Cloudflare Workers Documentation*.

Cloudflare. *Cloudflare Pages Documentation*.

Neon. *Neon Documentation*.

Google. *Gemini API Documentation*.